

Develop a "**Library Management System**" to demonstrate Object-Oriented Programming concepts.

Part A: Classes and Objects

Create a class named Book containing:

- Book ID
- Book Name
- Author
- Price

Create objects and display their details.

Part B: Constructors and Encapsulation

- Create both default and parameterized constructors.
- Declare data members as private.
- Access them using Getter and Setter methods.

Part C: Inheritance

Create the following classes:

- Person
- Student (inherits Person)
- Faculty (inherits Person)

Display the inherited properties using objects.

Part D: Polymorphism

Implement:

- Method Overloading for calculating the area of different shapes.
- Method Overriding by creating a Vehicle class and overriding the display() method in Car and Bike classes.

Part E: Abstraction and Interfaces

- Create an abstract class Shape containing an abstract method draw().
- Implement the draw() method in Circle and Rectangle classes.
- Create an interface Printable and implement it in a class named Report.

Algorithm: Library Program

Step 1: Start the program.

Step 2: Create a `Book` class with private data members:

- Book ID
- Book Name
- Author
- Price

Step 3: Create a constructor to initialize the book details.

Step 4: Create getter and setter methods to access and modify the private data.

Step 5: Create a `display()` method to display book details.

Step 6: Create a `Person` class with name and age.

Step 7: Create `Student` and `Faculty` classes that inherit from the `Person` class.

Step 8: Override the `display()` method in `Student` and `Faculty` using `super.display()`.

Step 9: Create an `Area` class and demonstrate method overloading using methods for:

- Circle area
- Rectangle area

Step 10: Create a `Vehicle` class with a `display()` method.

Step 11: Create `Car` and `Bike` classes and override the `display()` method.

Step 12: Create an abstract `Shape` class with an abstract `draw()` method.

Step 13: Create `Circle` and `Rectangle` classes that implement the `draw()` method.

Step 14: Create a `Printable` interface containing the `print()` method.

Step 15: Create a `Report` class that implements the `Printable` interface.

Step 16: In the `main()` method, create objects and demonstrate:

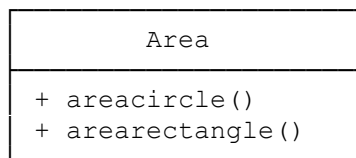
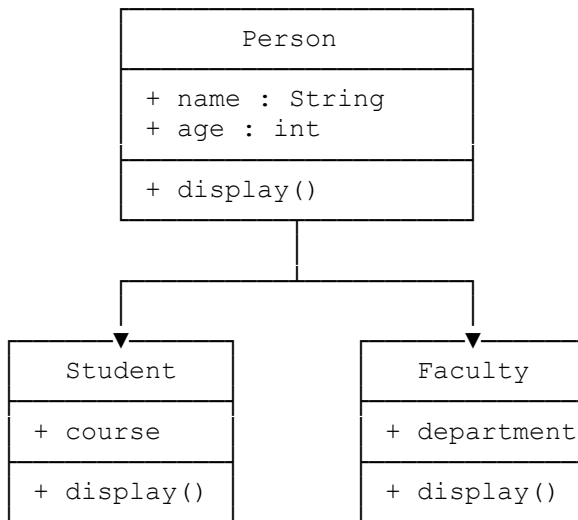
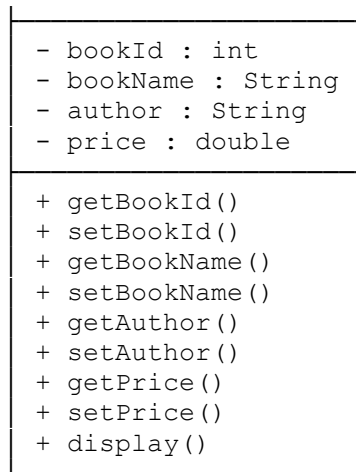
- Encapsulation
- Inheritance
- Method Overloading
- Method Overriding
- Abstraction
- Interface

Step 17: Display all results.

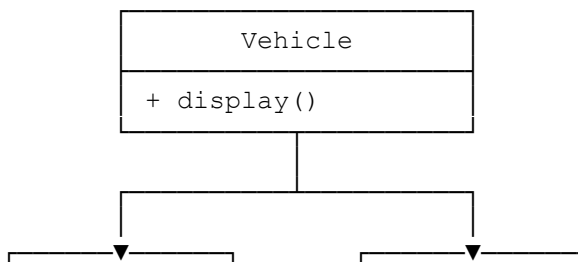
Step 18: Stop the program.

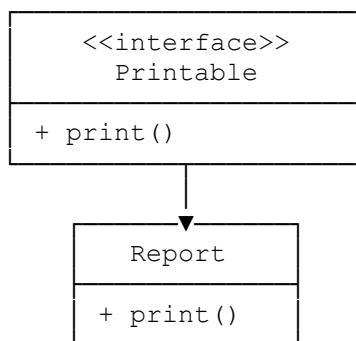
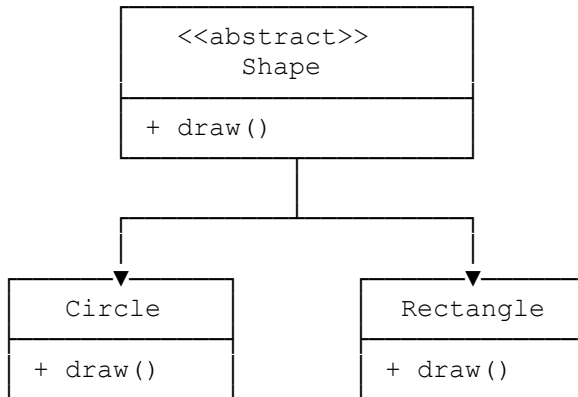
UML Class Diagram





Method
Overloading





Code:

```
package javacore;

class Book {
    private int bookId;
    private String bookName;
    private String author;
    private double price;
    Book(int id, String name, String auth, double pr) {
        bookId = id;
        bookName = name;
    }
}
```

```

        author = auth;
        price = pr;
    }
    public int getBookId() {
        return bookId;
    }
    public void setBookId(int id) {
        bookId = id;
    }
    public String getBookName() {
        return bookName;
    }
    public void setBookName(String name) {
        bookName = name;
    }
    public String getAuthor() {
        return author;
    }
    public void setAuthor(String auth) {
        author = auth;
    }
    public double getPrice() {
        return price;
    }
    public void setPrice(double pr) {
        price = pr;
    }
    public void display() {
        System.out.println("Book ID    : " + bookId);
        System.out.println("Book Name : " + bookName);
        System.out.println("Author   : " + author);
        System.out.println("Price    : " + price);
    }
}

// Part C: Inheritance
class Person {

```

```

    String name;
    int age;
    void display() {
        System.out.println("Name : " + name);
        System.out.println("Age : " + age);
    }
}
class Student extends Person {
    String course;
    void display() {
        super.display();
        System.out.println("Course : " + course);
    }
}
class Faculty extends Person {
    String department;
    void display() {
        super.display();
        System.out.println("Department : " + department);
    }
}
// Part D: Method Overloading
class Area {
    void areacircle(double radius) {
        System.out.println("Circle Area = " + (3.14 * radius *
radius));
    }
    void arearectangle(int length, int breadth) {
        System.out.println("Rectangle Area = " + (length *
breadth));
    }
}
// Part D: Method Overriding
class Vehicle {
    void display() {
        System.out.println("This is a vehicle.");
    }
}

```

```
    }  
}  
class Car extends Vehicle {  
    @Override  
    void display() {  
        System.out.println("This is a car.");  
    }  
}  
class Bike extends Vehicle {  
    @Override  
    void display() {  
        System.out.println("This is a bike.");  
    }  
}  
// Part E: Abstraction  
abstract class Shape {  
    abstract void draw();  
}  
class Circle extends Shape {  
    @Override  
    void draw() {  
        System.out.println("Drawing a Circle");  
    }  
}  
class Rectangle extends Shape {  
    @Override  
    void draw() {  
        System.out.println("Drawing a Rectangle");  
    }  
}  
// Part E: Interface  
interface Printable {  
    void print();  
}  
class Report implements Printable {  
    @Override
```

```

        public void print() {
            System.out.println("Printing Report...");
        }
    }
}

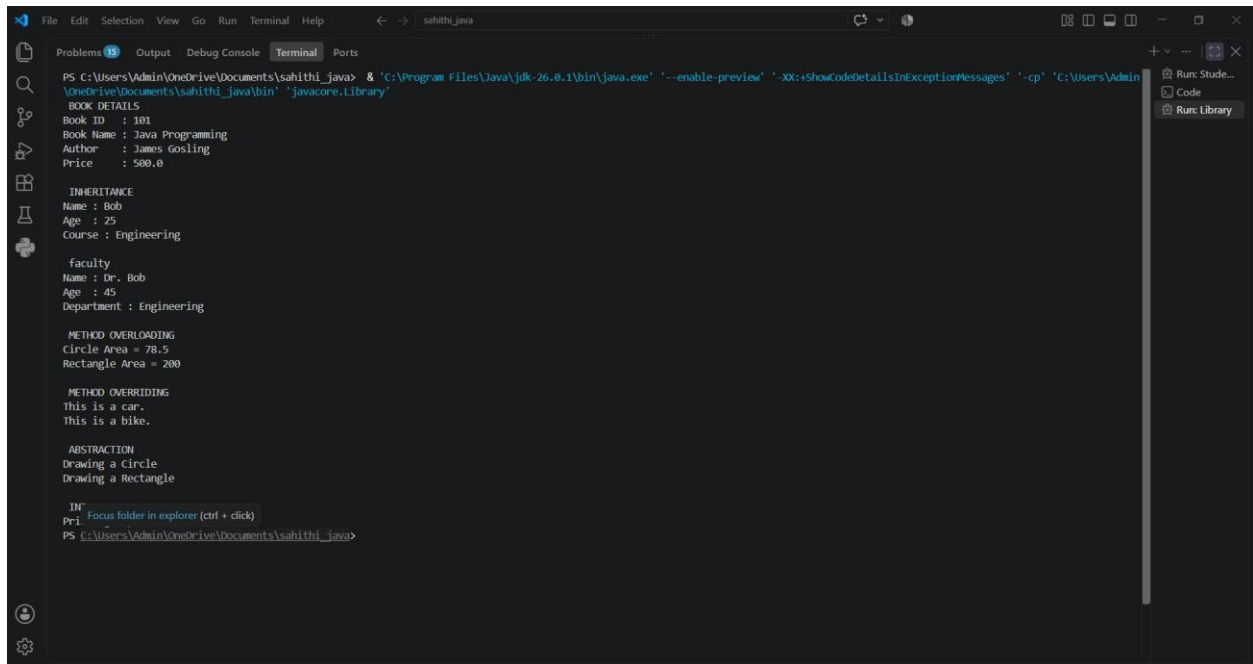
public class Library {
    public static void main(String[] args) {
        System.out.println(" BOOK DETAILS");
        Book b = new Book(101,"Java Programming","James
Gosling",500);
        b.display();
        System.out.println("\n INHERITANCE");
        Student s = new Student();
        s.name = "Bob";
        s.age = 25;
        s.course = "Engineering";
        s.display();
        System.out.println("\n faculty");
        Faculty f = new Faculty();
        f.name = "Dr. Bob";
        f.age = 45;
        f.department = "Engineering";
        f.display();
        System.out.println("\n METHOD OVERLOADING");
        Area a = new Area();
        a.areacircle(5.0);
        a.arearectangle(10, 20);
        System.out.println("\n METHOD OVERRIDING");
        Vehicle v1 = new Car();
        Vehicle v2 = new Bike();
        v1.display();
        v2.display();
        System.out.println("\n ABSTRACTION ");
        Shape sh1 = new Circle();
        Shape sh2 = new Rectangle();
        sh1.draw();
        sh2.draw();
    }
}

```



```
        System.out.println("\n INTERFACE ");
        Printable p = new Report();
        p.print();
    }
}
```

Code screenshot:



The screenshot shows an IDE window with a terminal pane. The terminal displays the output of a Java program. The output is as follows:

```
PS C:\Users\Admin\OneDrive\Documents\sahithi_java> & 'C:\Program Files\Java\jdk-26.0.1\bin\java.exe' '-enable-preview' '-XX:+ShowCodeDetailsInExceptionMessages' '-cp' 'C:\Users\Admin\OneDrive\Documents\sahithi_java\bin' 'javacore.library'
BOOK DETAILS
Book ID : 101
Book Name : Java Programming
Author : James Gosling
Price : 500.0

INHERITANCE
Name : Bob
Age : 25
Course : Engineering

faculty
Name : Dr. Bob
Age : 45
Department : Engineering

METHOD OVERLOADING
Circle Area = 78.5
Rectangle Area = 200

METHOD OVERRIDING
This is a car.
This is a bike.

ABSTRACTION
Drawing a Circle
Drawing a Rectangle

IN
Pri Focus folder in explorer (ctrl + click)
PS C:\Users\Admin\OneDrive\Documents\sahithi_java>
```